

Container Monitoring System with Message Queue Report

Group Members:
Yassin ES SAIM
Malik MOUSSA
Laurine Collin DUFRENSE
Nikita SIDARENKA
GitHub Repository

March 30, 2025

1 Introduction and Context

In modern distributed systems, effective monitoring is crucial for maintaining system reliability and performance. Our project implements a message queue-based system with two essential monitoring features that address common challenges in containerized environments.

2 Feature 1: Container Health Monitoring with Discord Alerts

2.1 Problem Statement and Motivation

In containerized environments, system failures can occur silently, leading to service disruptions that may go unnoticed. Traditional monitoring solutions often lack immediate notification capabilities or require constant dashboard observation. We identified the need for:

- Instant awareness of container failures
- Automated recovery procedures
- Historical tracking of system stability

2.2 Implementation Approach

Our solution leverages Discord's widespread adoption in development teams to create an effective alert system. The implementation consists of several interconnected components:

2.2.1 Alert Configuration

We implemented a Prometheus-based alert configuration that monitors container status:

```
1 apiVersion: 1
2 groups:
3   - name: ContainerAlerts
4     rules:
5       - uid: container_down
6         title: Container Down
7         condition: B
8         data:
9           - refId: A
10            datasourceUid: prometheus
11            model:
12              expr: up{job=~"app1|app2"} < 1
```

This configuration enables:

- Real-time status monitoring of critical containers
- Customizable alert conditions based on container metrics
- Integration with Grafana's alerting system

2.2.2 Discord Integration Architecture

The Discord notification system was designed with reliability in mind:

- **Webhook System:** Secure, token-based authentication
- **Message Format:** Structured alerts with severity levels and timestamps
- **Retry Logic:** Exponential backoff for failed notifications
- **Error Handling:** Comprehensive logging of notification failures

3 Feature 2: Message Queue Monitoring with Grafana and Prometheus

3.1 Problem Statement and Motivation

Message queues in distributed systems can become bottlenecks, with issues often discovered too late. Our Grafana dashboard addresses:

- Lack of visibility into message processing
- Difficulty in identifying performance bottlenecks
- Complex correlation between system metrics
- Need for historical performance analysis

3.2 Technical Implementation

3.2.1 Grafana and Prometheus Integration

We implemented a comprehensive monitoring stack that includes:

- **Prometheus:** For metrics collection, storage, and querying
- **Grafana:** For visualization and alerting capabilities

Our architecture allows real-time monitoring of all message queue components while maintaining historical data for trend analysis.

3.2.2 Dashboard Architecture

The Grafana dashboard provides comprehensive visualization of our message queue metrics:



Figure 1: Grafana Dashboard for Message Queue Monitoring

3.2.3 Prometheus Metrics Implementation

Our metrics collection system uses Spring's scheduling capabilities with Prometheus integration:

```
1 @Scheduled(fixedRate = 5000)
2 public void collectMetrics() {
3     // Message processing metrics with Prometheus collectors
4     messageCounter.increment();
5     processingTime.record(duration);
6     queueDepth.set(currentDepth);
7
8     // Prometheus-based error tracking
9     if (hasError) {
10         errorCounter.increment();
11         failureGauge.set(errorCount);
12         logErrorDetails(error); // Enhanced error logging
13     }
14 }
```

The integration with Prometheus allows us to:

- Create custom alerts based on complex conditions
- Track message processing rates and error percentages
- Monitor queue depths to prevent overflow conditions
- Correlate service performance with message processing metrics

4 Conclusion

The combination of container health monitoring and Grafana/Prometheus-based message queue visualization has created a robust monitoring solution. Key achievements include:

- Seamless integration of monitoring and alerting
- Real-time visibility into system performance
- Proactive issue detection and resolution
- Comprehensive system health management